

## 四. 全志Linux阶段

### 1. 为什么用高级ARM

嵌入式很多应用场景需要用到Linux系统，传统的SOC比如STM32F1系列是没有MMU内存管理单元的，无法支持Linux启动的虚拟内存映射的机制

高端ARM架构比如全志H616，瑞芯微的RV1126等平台代MMU单元，能运行Linux操作系统，在一些关于复杂应用程序，比如ROS系统，AI应用场景下

能得到软硬件的支持。

### 2. 什么是BSP，难度定位

BSP（Board Support Package），即板级支持包，具体点就是一个高端ARM芯片运行Linux系统的基本要素组成，

一般包含启动程序Bootloader，Linux内核，还有文件系统。

BSP开发工程师是嵌入式软件工程师的一个细分领域，负责板级支持包的开发、调试和维护工作。

他们需要既懂硬件工作原理，又能编写对应的软件使之正常工作，

技能综合性相对较高，学习和开发难度较大，但待遇也相对较好。刚入行的新手不太适合，相应的新手也不会匹配较好的待遇，一般这种岗位在一线城市的

起步年薪不低于30万，可以作为嵌入式新人未来的发展方向，随着国内嵌入式技术的发展和局势，一般芯片原厂招聘这类型岗位，对学校学历也有一些要求。

#### 2.1 全志H616的Bootloader编译流程

首先，将Bootloader源码和工具链复制到新的文件夹下。

然后，在终端中进入Bootloader目录，

使用 `make` 命令配合 `ARCH` 和 `CROSS_COMPILE` 参数进行清理工程、生成.config文件以及编译Bootloader。

编译完成后，会生成用于烧写的文件，如 `sunxi-spl.bin` 和 `u-boot.itb`。

最后，使用 `dd` 命令将这些文件烧写到SD卡中，完成Bootloader的编译和烧写过程。

#### 2.2 全志H616的Linux内核编译流程

首先，从Linux内核官网下载主线内核源码并解压。

接着，在内核源码目录下，

使用 `make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig` 命令来生成默认配置文件。

然后，通过 `make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- menuconfig` 命令进入图形化配置界面，根据需求进行内核配置裁剪。

配置完成后，执行 `make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j16` 命令来编译内核，

其中 `-j16` 表示使用16个线程进行编译，可以根据实际CPU核心数调整该值。最后，编译生成的内核映像文件可用于后续的系统安装或测试。

## 2.4 全志H616的根文件系统

首先，在Buildroot配置界面中选择文件系统类型，通常为ext4格式，并设置根文件系统的大小，例如128M。

接着，使用fdisk工具在TF卡上创建新的分区，并格式化为ext4文件系统。

然后，将编译好的根文件系统镜像（如rootfs.tar）复制到TF卡的相应分区中，并解压缩。

最后，在U-Boot中设置启动命令和bootargs变量，以确保内核能够正确引导并挂载根文件系统。

这样，全志H616的根文件系统就制作完成了，可以用于系统的启动和运行。

## 3. ARM+Linux的外设和单片机有何区别

驱动一个外设，不管在外设通信协议以及外设通信数据上是没有区别的，比如在STM32上面获取温湿度DHT11的数据，和在全志H616加Linux这样架构下去获取

DHT11的场景对比中，关于DHT11的外设接线及数据交互是没有区别的，只要区别在处理器端，STM32裸板程序直接通过寄存器操作或者通过HAL库间接操作

寄存器，就可以和DHT11交互数据，而全志H616加Linux的架构，首先需要遵守Linux操作系统的开发原则，把程序分为应用层和驱动层，驱动层除了和STM32

类似的寄存器，时序图操作之外，还要遵守Linux设备驱动框架，这样Linux系统才能识别DHT11的设备，

并在文件系统的/dev/目录下生成一个设备驱动节点文件，

应用层的程序访问硬件就像访问普通文件一样，向内核的驱动程序发起数据请求，获得DHT11的数据。

## 4. 设备驱动总结

### 4.1 设备驱动定位

Linux设备驱动包含字符设备驱动，块设备驱动及网络设备驱动，近年来更加流行通用的是设备树。

我们课程培养定位方向偏向于应用，目前也会讲解设备驱动程序的开发，使用Module\_INIT宏来做设备程序框架入口，能实现基本的字符设备驱动的开发。

也是因为嵌入式Linux方向的现状更偏向于应用，综合难度，工程师等级，时间成本等综合考虑才会减少驱动内容。

如果入职后分配到驱动岗位，也具备快速上手学习培养的能力。

### 4.2 字符设备开发流程

为Linux编写字符设备驱动程序是一个涉及内核编程的过程，以下是基本步骤和概念的概述：

#### 1. 定义设备号：

- 为字符设备分配一个主设备号（major number），这通常需要向内核维护者申请。
- 定义次设备号（minor number），用于区分同一驱动支持的不同设备。

#### 2. 编写驱动程序：

- **初始化驱动**：定义一个 `init` 函数，在驱动加载时被调用，用于初始化驱动程序。
- **打开和关闭设备**：实现 `open` 和 `release` 函数，分别在设备被打开和关闭时调用。

- **读写操作**：实现 `read` 和 `write` 函数，用于处理对设备的读写请求。
- **控制操作**：如果设备支持 `ioctl` 调用来控制设备，需要实现 `ioctl` 函数。
- **文件操作结构体**：定义一个 `file_operations` 结构体，将上述函数指针关联到对应的操作。

### 3. 注册字符设备：

- 使用 `register_chrdev_region` 或 `alloc_chrdev_region` 函数注册设备号和驱动。
- 使用 `cdev_init` 初始化 `cdev` 结构体，并将 `file_operations` 结构体传递给它。
- 使用 `cdev_add` 将 `cdev` 结构体添加到内核中。

### 4. 模块加载和卸载：

- 实现 `init_module` 和 `cleanup_module` 函数，分别作为模块加载和卸载时的入口点。

### 5. 错误处理：

- 在驱动程序中添加必要的错误处理代码，确保在发生错误时能够正确响应。

### 6. 编译驱动程序：

- 编写 `Makefile`，确保驱动程序能够与内核一起编译。
- 使用 `make` 命令编译驱动程序，生成模块文件。

### 7. 加载和测试驱动程序：

- 使用 `insmod` 命令加载驱动模块。
- 使用 `dmesg` 查看内核日志，确认驱动程序加载成功。
- 使用 `mknod` 创建设备节点，例如 `/dev/mychardev`。
- 使用 `chmod` 设置设备节点的权限。
- 使用 `cat`、`echo` 等命令测试设备的读写功能。

### 8. 卸载驱动程序：

- 使用 `rmmmod` 命令卸载驱动模块。
- 确保所有资源都被正确释放，包括设备节点。

### 9. 调试和优化：

- 使用 `printk`、`kdebug` 等工具进行调试。
- 根据需要优化驱动程序的性能。

## 5. 全志Linux的超声波实现方案

### 5.1 面试过程重点定位

**注意：**这里能够吸引面试官的部分，是IO口驱动的自己实现，我们使用wiringPi库没有问题，那么这个功能在现场面试的时候，重点变成了时间处理函数，

可能说服力不强，实际上课程里讲解了字符设备驱动，可以引导面试官把对于这部分的探讨定位在驱动开发流程，前提，课程相应内容要熟悉！！

如果引导到驱动，那么我们的思路是在底层驱动上只是实现了高低电平的输出功能，而超声波的时序处理流程是放在应用层来做的，说白了，

引导面试官跟你去做类似于`digitalwrite(13, HIGH);`的具体实现方式上，就有更好的印象

## 5.2 内容回顾

超声波测距的原理和单片机阶段是一样的，而Linux应用层关于时间的控制区别于单片机的定时器。

对于基本IO口的访问，属于字符设备驱动的范畴，如果面试过程中面试官要你阐述Linux字符设备驱动开发，我们要去掌握课程中关于内核交叉编译，设备

驱动框架的内容。我也可以在这里简述

**Linux字符设备开发流程主要包括：定义设备号、编写驱动程序框架（包括初始化、打开、关闭、读/写等操作）、**

**注册字符设备驱动到内核、编译并加载驱动模块、进行设备操作测试**

更详细的步骤总结参考4.2小节

那么关于超声波时序部分的时间管理是用时间处理函数gettimeofday

完整代码回顾

```
#include <stdio.h>
#include <sys/time.h>
#include <wiringPi.h>
#include <stdlib.h>
#include <unistd.h>

#define Trig 0
#define Echo 1

double getDistance()
{
    double dis;

    struct timeval start;
    struct timeval stop;

    pinMode(Trig, OUTPUT);
    pinMode(Echo, INPUT);

    digitalWrite(Trig ,LOW);
    usleep(5);

    digitalWrite(Trig ,HIGH);
    usleep(10);
    digitalWrite(Trig ,LOW);
    /*above init CSB*/

    while(!digitalRead(Echo));
    gettimeofday(&start,NULL);
    while(digitalRead(Echo));
    gettimeofday(&stop,NULL);

    long diffTime = 1000000*(stop.tv_sec-start.tv_sec)+(stop.tv_usec -
start.tv_usec);
    printf("diffTime = %ld\n",diffTime);
    dis = (double)diffTime/1000000 * 34000 / 2;
```

```

        return dis;
    }

    int main()
    {
        double dis;

        if(wiringPiSetup() == -1){
            fprintf(stderr,"%s","initwringPi error");
            exit(-1);
        }

        while(1){

            dis = getDistance();
            printf("dis = %lf\n",dis);
            usleep(500000);
        }

        return 0;
    }

```

### 5.3 综合表述Linux超声波测距功能实现

程序首先初始化 `wiringPi` 库（这里去引导自己写驱动的内容），然后进入一个无限循环，在循环中调用 `getDistance` 函数来获取距离。

该函数通过设置Trig引脚为输出，Echo引脚为输入，先发送一个10微秒的高电平信号触发超声波发射，然后等待Echo引脚变为高电平，记录开始时间，再等待Echo引脚变为低电平，记录结束时间。

通过计算这两个时间点之间的差值（以微秒为单位），

并根据声波在空气中的速度（约340米/秒），计算出超声波往返的时间，最后除以2得到距离值（单位：厘米）。

主循环中，每次测量后程序会打印出测量到的距离，并等待半秒再次进行测量。

## 6.全志Linux关于SG90舵机的开发

实现的方式是模拟PWM信号，借助GPIO口和Linux系统的软件定时，应用的知识点等同于超声波测距。

### 6.1 Linux软件定时器回顾

分析：实现定时器，通过 `itimerval` 结构体以及函数 `setitimer` 产生的信号，系统随之使用 `signal` 信号处理函数来处理产生的定时信号。从而实现定时器。

先看 `itimerval` 的结构体

```

struct itimerval
{
    /* value to put into `it_value' when the timer expires. */
    struct timeval it_interval;
    /* Time to the next timer expiration. */
    struct timeval it_value;
}

```

```
};
```

**it\_interval**: 计时器的初始值，一般基于这个初始值来加或者来减，看控制函数的参数配置

**it\_value:** 程序跑到这之后，多久启动定时器

```
struct timeval
{
    __time_t tv_sec;        /* Seconds. */
    __suseconds_t tv_usec;  /* Microseconds. */
};

int setitimer (__itimer_which_t __which,
               const struct itimerval *__restrict __new,
               struct itimerval *__restrict __old)
```

setitimer()将value指向的结构体设为计时器的当前值，如果ovalue不是NULL，将返回计时器原有值。

which:三种类型

ITIMER\_REAL //数值为0, 计时器的值实时递减, 发送的信号是SIGALRM。

ITIMER\_VIRTUAL //数值为1, 进程执行时递减计时器的值, 发送的信号是SIGVTALRM。

ITIMER\_PROF //数值为2, 进程和系统执行时都递减计时器的值, 发送的信号是SIGPROF。

很明显，这边需要捕获对应的信号进行逻辑相关处理 `signal(SIGALRM,signal_handler);`

返回说明：

成功执行时，返回0。失败返回-1

实现代码：

```
/*该代码实现的功能是： 1s后开启定时器，然后每隔1s向终端打印hello。*/  
#include <stdio.h>  
#include <sys/time.h>  
#include <stdlib.h>  
#include <signal.h>  
  
static int i;  
  
void signal_handler(int signum)  
{  
    i++;  
    if(i == 2000){  
        printf("hello\n");  
        i = 0;  
    }  
}  
  
int main()  
{  
    struct itimerval itv;  
    //!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! 这里对应课程部分的代码，两个时间戳是做反了，参  
    考下节课!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!  
    //设定定时时间  
    itv.it_interval.tv_sec = 0;  
    itv.it_interval.tv_usec = 500;
```

```

//设定开始生效，启动定时器的时间
itv.it_value.tv_sec = 1;
itv.it_value.tv_usec = 0;

//!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! 这里对应课程部分的代码，两个时间戳是做反了，参考下节课!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

//设定定时方式
if( -1 == setitimer(ITIMER_REAL, &itv, NULL)){
    perror("error");
    exit(-1);
}

//信号处理
signal(SIGALRM,signal_handler);

while(1);

return 0;
}

```

这种方法需要注意的是，一个进程只能创建一个定时器

## 6.2 舵机控制实现过程回顾

我们可以回顾下课程代码

```

#include <stdio.h>
#include <sys/time.h>
#include <stdlib.h>
#include <signal.h>
#include <wiringPi.h>

#define SG90Pin 5

int jd;

static int i = 0;

void signal_handler(int signum)
{
    if(i <= jd){
        digitalWrite(SG90Pin, HIGH);
    }else{
        digitalWrite(SG90Pin, LOW);
    }

    if(i == 40){
        i = 0;
    }
    i++;
}

int main()
{
    struct itimerval itv;
    jd = 0;
}

```

```

wiringPiSetup();
pinMode(SG90Pin, OUTPUT);

//设定定时时间
itv.it_interval.tv_sec = 0;
itv.it_interval.tv_usec = 500;
//设定开始生效，启动定时器的时间
itv.it_value.tv_sec = 1;
itv.it_value.tv_usec = 0;
//设定定时方式
if( -1 == setitimer(ITIMER_REAL, &itv, NULL)){
    perror("error");
    exit(-1);
}
//信号处理
signal(SIGALRM, signal_handler);

while(1){
    printf("input jd: 1-0 2-45 3-90 4-135 \n");
    scanf("%d",&jd);
}

return 0;
}

```

这里用到Linux系统编程中关于信号的处理。

## 6.3 面试官问你是如何在Linux下控制舵机

程序首先初始化 `wiringPi` 库和舵机所连接的GPIO引脚为输出模式，

后期也尝试自己实现了GPIO口的驱动（需要你对课程中字符设备的内容比较了解，参考超声波那块，尽量要这样引导到驱动的探讨）

然后设置一个定时器，每隔500微秒触发一次 `SIGALRM` 信号。

信号处理函数 `signal_handler` 根据全局变量 `i` 的值来控制舵机的高低电平，模拟输出PWM的信号，从而控制舵机转动到不同的角度。

主循环中，程序等待用户输入不同的 `jd` 值，这些值代表舵机需要转动到的角度（1-0度，2-45度，3-90度，4-135度），并根据输入值来控制舵机的转动。

## 7. Linux串口开发

### 7.1 面试官问，Linux下如何开发串口

Linux应用层串口开发通常涉及使用系统调用来打开、配置、读写和关闭串口设备文件，

如 `/dev/ttyS0` 或 `/dev/ttyUSB0`，通过设置串口参数（波特率、数据位、停止位等），

然后通过读写操作与串口设备进行数据交换，实现与外部设备的通信。



## 7.2 面试官问，串口配置是什么结构体

在Linux应用层串口开发中，配置串口通常使用 `termios` 结构体，

该结构体包含了控制串口行为的所有选项，如波特率、字符大小、奇偶校验、停止位等，

通过 `tcgetattr` 和 `tcsetattr` 函数分别获取和设置串口的 `termios` 结构体参数，从而实现对串口的配置。

## 7.3 Linux串口开发和STM32有什么不同

Linux串口开发与STM32串口开发的主要区别在于开发环境、编程接口和配置方式。

Linux串口开发通常使用 `termios` 结构体来配置串口参数，如波特率、数据位、停止位等，

并通过系统调用如 `open`、`read`、`write` 和 `close` 来操作串口设备文件（例如 `/dev/ttyS0`）。

而STM32串口开发则依赖于硬件抽象层（HAL）库或直接操作寄存器来配置串口参数，

并通过库函数如 `HAL_UART_Transmit` 和 `HAL_UART_Receive` 来发送和接收数据。

STM32开发更接近硬件层面，需要考虑时钟配置、GPIO设置等硬件相关的初始化步骤，

而Linux开发则更侧重于软件层面的接口调用和数据处理。

## 7.4 代码回顾

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <stdarg.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/types.h>
#include <sys/stat.h>

#include "wiringSerial.h"

int myserialOpen (const char *device, const int baud)
{
    struct termios options ;
    speed_t myBaud ;
    int status, fd ;

    switch (baud){
        case 9600: myBaud = B9600 ; break ;
        case 115200: myBaud = B115200 ; break ;
    }
    if ((fd = open (device, O_RDWR | O_NOCTTY | O_NDELAY | O_NONBLOCK)) == -1)
        return -1 ;

    fcntl (fd, F_SETFL, O_RDWR) ;

    // Get and modify current options:
```

```

    tcgetattr (fd, &options) ;

    cfmakeraw (&options) ;
    cfsetispeed (&options, myBaud) ;
    cfsetospeed (&options, myBaud) ;

    options.c_cflag |= (CLOCAL | CREAD) ;
    options.c_cflag &= ~PARENB ;
    options.c_cflag &= ~CSTOPB ;
    options.c_cflag &= ~CSIZE ;
    options.c_cflag |= CS8 ;
    options.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG) ;
    options.c_oflag &= ~OPOST ;

    options.c_cc [VMIN] = 0 ;
    options.c_cc [VTIME] = 100 ;    // Ten seconds (100 deciseconds)

    tcsetattr (fd, TCSANOW, &options) ;

    ioctl (fd, TIOCMGET, &status);

    status |= TIOCM_DTR ;
    status |= TIOCM_RTS ;

    ioctl (fd, TIOCMSET, &status);

    usleep (10000) ;    // 10ms

    return fd ;
}

void serialSendstring (const int fd, const char *s)
{
    int ret;
    ret = write (fd, s, strlen (s));
    if (ret < 0)
        printf("Serial Puts Error\n");
}

int serialGetString (const int fd, char *buffer)
{
    int n_read;

    n_read = read(fd, buffer, 32);

    return n_read;
}

```

## 7.5 串口其他具体知识

回顾并参考STC51里面关于串口的描述，在串口底层的内容无论是H616加Linux,还是STM51或者STM32都是一样的。

涉及到串口协议，波特率，串口速度优化等。

## 8. UDEV机制

### 8.1 面试官问什么是守护进程

守护进程（Daemon）是一种在后台运行的特殊进程，它通常在系统启动时自动启动，并在后台提供系统或网络服务，如监听网络请求、处理任务队列、管理硬件设备等，它们通常不与用户直接交互，而是作为服务提供者在后台默默工作，确保系统的正常运行和功能实现。

### 8.2 如何开发一个守护进程

直接借助daemon()函数完成。

代码回顾：

调用 daemon 函数创建一个守护进程，如果创建失败则打印错误信息并退出。创建后为它设置了一个信号处理函数 handler 来处理 SIGQUIT 信号，当接收到该信号时，会打印一条消息并将全局变量 flag 设置为 false，从而允许循环退出。在守护进程的主循环中，它每隔10秒打开日志文件 /home/orangepi/daemon.log，将当前时间写入文件，然后关闭文件描述符，直到接收到 SIGQUIT 信号，flag 变为 false，循环结束，守护进程随之退出。这段代码展示了守护进程的基本框架，包括创建守护进程、处理信号和执行周期性任务

```
#include <unistd.h>
#include <signal.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <time.h>
#include <stdio.h>

static bool flag = true;

void handler(int sig)
{
    printf("I got a signal %d\nI'm quitting.\n", sig);
    flag = false;
}

int main()
{
    time_t t;
    int fd;
    //创建守护进程
    if(-1 == daemon(0, 0))
    {
        printf("daemon error\n");
    }
}
```

```

        exit(1);
    }

    //设置信号处理函数
    struct sigaction act;
    act.sa_handler = handler;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    if(sigaction(SIGQUIT, &act, NULL))
    {
        printf("sigaction error.\n");
        exit(0);
    }
    //进程工作内容
    while(flag)
    {
        fd = open("/home/orangepi/daemon.log", O_WRONLY | O_CREAT | O_APPEND,
0644);
        if(fd == -1)
        {
            printf("open error\n");
        }
        t = time(0);
        char *buf = asctime(localtime(&t));
        write(fd, buf, strlen(buf));
        close(fd);
        sleep(10);
    }
    return 0;
}

```

## 8.3 什么是UDEV机制

UDEV (User-space Device Manager) 是一个运行在用户空间的守护进程

UDEV机制是Linux系统中用于热插拔，它能够根据设备的特性生成相应的设备文件，并允许用户通过规则来管理这些设备文件。

规则文件是 udev 里最重要的部分

udev 规则的匹配键

**ACTION:** 事件 (uevent) 的行为，例如: add (添加设备)、remove (删除设备);

**KERNEL:** 内核设备名称，例如: sda, cdrom;

**DEVPATH:** 设备的 devpath 路径;

**SUBSYSTEM:** 设备的子系统名称，例如: sda 的系统为 block;

**BUS:** 设备在 devpath 里的总线名称，例如: usb;

**DRIVER:** 设备在 devpath 的设备驱动名称，例如: ide-cdrom;

**ID:** 设备在 devpath 里的识别号;

**SYSFS{filename}:** 设备的 devpath 路径下，设备的属性文件 "filename" 里的内容;

## 8.4 UDEV自动挂载U盘

```
ACTION=="add", SUBSYSTEMS=="usb", SUBSYSTEM=="block", RUN{program}+="/bin/mkdir /media/%k", RUN{program}+="/usr/bin/systemd-mount --no-block --collect $devnode /media/%k"
```

## 8.5 语音刷抖音-面试聊的重点

这称不上一个项目，和面试官聊这部分内容更多是引导出你对UDEV的理解，完成手机插入Linux系统后自动加载的能力。

当然也是Linux串口开发的应用，因为语音模块的识别结果是通过串口传给Linux的。

那么语音识别之后关于对手机的控制主要是adb shell的指令，编程实现读取串口数据，并根据输出处理后执行adbshell指令，完成手机上划，下划，点赞等功能。

```
adb shell input swipe 540 1300 540 500 100 向下滑动540是水平的，1300是竖直方向，下 是 500
adb shell input swipe 540 500 540 1300 100 向上滑动
adb shell "seq 3 | while read i;do input tap 350 1050 & input tap 350 1050 & sleep 0.01;done;" 点赞
adb shell input keyevent 26 锁屏
```

手机接入的热插拔回顾

- 把手机接入开发板
- 安装adb工具，在终端输入adb安装指令：`sudo apt-get install adb`
- dmeg能看到手机接入的信息，但是输入adb devices会出现提醒  
insufficient permissions for device: user in plugdev group; are your udev rules wrong?
- 配置文件，以支持USB设备的热插拔，支持UDEV的机制  
在/etc/udev/rules.d 文件夹下创建规则文件  
`cd /etc/udev/rules.d/`  
`sudo vim 51-android.rules`  
在文件中添加内容 `SUBSYSTEM=="usb", ENV{DEVTYPE}=="usb_device", MODE="0666"`
- 在手机开发者选项中，打开USB调试，重新拔插手机
- 手机弹出调试提醒，点确认手机调试模式

## 9. 嵌入式数据库SQLite

### 9.1 面试过问你用过哪些数据库

嵌入式常用数据库是SQLite,

是轻量化，易用的嵌入式数据库，用于设备端的数据管理，可以理解成单点的数据库。传统服务器型数据库用于管理多端设备，更加复杂常用数据。

### 9.2 数据库指令-要复习一下

- 创建一个数据库

方式一：

```
1. sqlite3 进入数据库
2. .open test.db
3. .quit
数据库退出后在命令当前路径创建数据库test.db
```

方式二:

```
sqlite3 test.db //在命令运行当前窗口创建数据库test.db
在数据库命令下
.databases 列出当前打开的数据库
.quit 退出
```

- **创建一张表格**

```
create table stu2(id Integer,name char,score Integer);
```

- **插入一条记录**

```
insert into stu values(18130106,'huang',99);
insert into stu2 values(18130101,"gang",100);      ''和""都行
insert into stu(name,score) values("huanggang",98);  插入部分字段内容
```

- **查看数据库的记录**

```
select * from stu; //查询所有字段的结果
select name,score from stu; //查询数据库中部分字段的内容
```

- **删除一条记录**

```
delete from stu where id = 18130101;
```

- **更改一条记录**

```
update stu set name = 'huangg' where id = 18130106;
```

- **删除一张表**

```
drop table stu;
```

- **增加一列**

```
alter table stu add column sex char;
```

## 9.3 SQLite的编程操作

C语言对于SQLite的数据库操作主要有3个API，打开，关闭，执行数据库指令

```
sqlite3_open(const char *filename, sqlite3 **ppDb) //打开

sqlite3_close(sqlite3*) 该例程关闭之前调用 sqlite3_open() //关闭

sqlite3_exec(); //执行数据库操作指令
```

## 10.交叉编译

### 10.1 什么是交叉编译

交叉编译是一种软件编译过程，其中源代码在一个平台上编写，但编译成的目标代码是为另一个不同的平台准备的。

这个过程涉及到使用一个平台的编译器来生成另一个平台的可执行代码，通常是为了在嵌入式系统或不同架构的硬件上运行程序。

交叉编译器包含了目标平台的库和头文件，确保生成的程序能够正确地在目标硬件上运行。

### 10.2 如何安装交叉编译工具

安装交叉编译工具通常包括以下步骤：

首先，确认系统环境是否满足安装要求。

然后，下载适合目标架构的交叉编译工具链，例如可以从Linaro官网下载ARM架构的交叉编译器

。接着，将下载的文件解压到指定目录，例如 `/usr/local/arm`，并确保该目录具有正确的权限。

最后，通过修改系统的环境变量，如 `PATH`，来添加交叉编译工具的路径，使得系统能够识别并使用新的交叉编译器。完

成这些步骤后，重启系统以使环境变量的更改生效，并通过编译一个简单的程序来验证交叉编译器是否安装成功。

### 10.2 你用的交叉编译工具链版本

这个问题有点坑，可以记住：

香橙派Zero2的交叉编译工具链版本是aarch64-linux-gnu-gcc 11.2-2022.02，

这是基于GCC 11.2的GNU Toolchain，适用于Arm架构，包括bare-metal和Linux工具链。

## 11. Makefile

`Makefile` 是一个用于管理和自动化软件构建过程的文件，

它与 `make` 工具一起使用。`make` 工具会查找 `Makefile` 中的指令来决定如何编译和链接程序。

以下是 `Makefile` 的一些基本概念和组成部分：

### 11.1 基本结构

1. **目标 (Targets)**：通常是文件名，`make` 会寻找这些文件并根据依赖关系决定是否重新构建。
2. **依赖 (Dependencies)**：目标文件依赖的文件列表。
3. **规则 (Rules)**：如何从依赖生成目标的命令。

## 11.2 基本语法

```
target: dependencies
      command
```

- `target`：目标文件。
- `dependencies`：目标文件依赖的文件列表，如果这些文件比目标文件新，则 `make` 会执行后面的命令来更新目标文件。
- `command`：生成目标文件的命令，每条命令必须以 `tab` 字符开始。

## 11.3 示例

```
# 这是一个注释

all: hello.o world.o

hello.o: hello.c
      gcc -c hello.c

world.o: world.c
      gcc -c world.c

clean:
      rm -f *.o
```

在这个例子中，`all` 是一个伪目标，它依赖于 `hello.o` 和 `world.o`。`make` 默认会寻找名为 `all` 的目标，或者直接指定目标，如 `make hello.o`。

## 11.4 变量

```
CC=gcc
CFLAGS=-Wall

all: hello.o world.o

hello.o: hello.c
      $(CC) $(CFLAGS) -c hello.c
```

在这里，`CC` 和 `CFLAGS` 是变量，可以在 `Makefile` 中定义并在命令中使用 `$( )` 引用。

## 11.5 模式规则

```
%.o: %.c
      $(CC) $(CFLAGS) -c $<

all: hello.o world.o

clean:
      rm -f *.o
```

`%` 是一个通配符，可以匹配任何字符串。这使得 `Makefile` 更加简洁，因为你可以为所有 `.o` 文件编写一个通用的规则。



## 11.6 伪目标

- `.PHONY`：声明一个伪目标，即使当前目录下存在同名文件，`make` 也会执行这个目标的命令。
- `all`：默认目标，如果没有指定目标，`make` 会构建这个目标。
- `clean`：常用于清理项目，删除所有编译生成的文件。

## 11.7 包含其他 Makefile

```
include another_makefile
```

这允许你将构建过程分解到多个文件中，提高 Makefile 的可读性和可维护性。

Makefile 是一个强大的工具，可以帮助你自动化构建过程，减少重复劳动，并确保构建的一致性。学习如何编写有效的 Makefile 对于任何 C/C++ 项目都是非常重要的。

# 12. C和Python混合开发

## 12.1 C语言如何调用python

在C语言中调用Python代码通常涉及到使用Python的C API。以下是一些基本步骤和概念：

### 1. 包含Python头文件：

在你的C代码中，你需要包含Python的头文件，通常是 `Python.h`。

### 2. 初始化Python解释器：

使用 `Py_Initialize()` 函数初始化Python解释器。

### 3. 导入Python模块：

使用 `PyImport_ImportModule()` 函数导入你想要调用的Python模块。

### 4. 获取模块中的函数：

使用 `PyObject*` 类型和 `PyObject_GetAttrString()` 函数获取模块中特定函数的引用。

### 5. 调用Python函数：

使用 `PyObject_CallObject()` 或 `PyObject_CallFunction()` 等函数调用Python函数。

### 6. 处理返回值：

调用Python函数后，你将得到一个 `PyObject*` 类型的返回值，可能需要使用

`PyArg_ParseTuple()` 等函数将其转换为C类型的值。

### 7. 清理：

使用 `Py_DECREF()` 或 `Py_XDECREF()` 管理引用计数，并在结束时调用 `Py_Finalize()` 来关闭Python解释器。

下面是一个简单的例子，展示了如何在C程序中调用一个简单的Python函数：

```
#include <Python.h>

int main() {
    // 初始化Python解释器
    Py_Initialize();

    // 导入名为"mymodule"的模块
    PyObject *pName = PyUnicode_FromString("mymodule");
    PyObject *pModule = PyImport_Import(pName);
    Py_DECREF(pName);
```

```

if (pModule != NULL) {
    // 获取模块中的函数"myfunction"
    PyObject *pFunc = PyObject_GetAttrString(pModule, "myfunction");
    if (pFunc && PyCallable_Check(pFunc)) {
        // 调用函数
        PyObject *pArgs = PyTuple_New(0); // 无参数
        PyObject *pValue = PyObject_CallObject(pFunc, pArgs);
        Py_DECREF(pArgs);

        if (pValue != NULL) {
            // 处理返回值
            printf("Result of call: %ld\n", PyLong_AsLong(pValue));
            Py_DECREF(pValue);
        } else {
            Py_DECREF(pFunc);
            Py_DECREF(pModule);
            PyErr_Print();
            fprintf(stderr, "Call failed\n");
            return 1;
        }
    } else {
        if (PyErr_Occurred())
            PyErr_Print();
        fprintf(stderr, "Cannot find function \"myfunction\"\n");
    }
    Py_XDECREF(pFunc);
    Py_DECREF(pModule);
} else {
    PyErr_Print();
    fprintf(stderr, "Failed to load \"mymodule\"\n");
    return 1;
}

// 关闭Python解释器
Py_Finalize();

return 0;
}

```

在这个例子中，我们假设有一个名为 `mymodule.py` 的Python模块，其中包含一个名为 `myfunction` 的函数。C程序将调用这个函数并打印其返回值。

请注意，为了使这个C程序能够工作，你需要确保Python头文件和库文件的路径被正确地添加到编译器的包含路径和链接器的库路径中。这通常涉及到在编译命令中添加 `-I` 和 `-L` 选项，并链接Python库（例如 `-lpython3.x`，其中 `3.x` 是Python版本号）。

## 13. 其他

远程登录工具，文件传输工具，固件更新，刷机流程，网络配置等就不展开了，平常我们常用的东西如果面试提起，正常发挥就行。

## 14. 全志Linux项目描述放在第七章